11.3 WebSocket 服务器实现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~160,000 tokens，输出 ~12,000 tokens（本章合计）

WebSocket 是 Gateway 的核心传输协议。所有实时通信——客户端控制、Agent 事件流、节点命令——都通过 WebSocket 进行。本节深入分析其实现。

11.3.1 WebSocket 传输层： `ws` 库的使用与配置

OpenClaw 使用 ws 库作为 WebSocket 服务器实现。 `ws` 是 Node.js 生态中最成熟的 WebSocket 库，以高性能和低开销著称。

WebSocket 服务器以 `noServer` 模式创建——它不自己监听端口，而是挂载在 HTTP 服务器上，通过 HTTP Upgrade 请求建立连接：

```
// 概念示意（src/gateway/server.impl.ts）
import { WebSocketServer } from "ws";
import { createServer as createHttpServer } from "node:http";

const httpServer = createHttpServer();
const wss = new WebSocketServer({ noServer: true });

// HTTP Upgrade 请求由 HTTP 服务器转发给 WebSocket 服务器
httpServer.on("upgrade", (request, socket, head) => {
  // 根据 URL 路径决定是否升级为 WebSocket
  // - / → Gateway WS
  // - /canvas → Canvas WS
  wss.handleUpgrade(request, socket, head, (ws) => {
    wss.emit("connection", ws, request);
  });
});
```

`noServer` 模式的好处是 HTTP 和 WebSocket 可以共享同一个端口。 `/v1/chat/completions` 这样的 HTTP 路径由 HTTP 处理器处理，而根路径 `/` 的 WebSocket Upgrade 请求交给 WebSocket 服务器。

11.3.2 WebSocket 运行时管理

`src/gateway/server-ws-runtime.ts` 是 WebSocket 运行时的入口：

```
export function attachGatewayWsHandlers(params: {
  wss: WebSocketServer;
  clients: Set<GatewayWsClient>;
  port: number;
  resolvedAuth: ResolvedGatewayAuth;
  gatewayMethods: string[];
  events: string[];
  extraHandlers: GatewayRequestHandlers;
  broadcast: (event: string, payload: unknown, opts?) => void;
  context: GatewayRequestContext;
  // ...日志器
}) {
  attachGatewayWsConnectionHandler({ ...params });
}
```

这个函数将 WebSocket 连接处理器绑定到 WebSocket 服务器。 `clients` 是一个 `Set<GatewayWsClient>` ，保存所有活跃的客户端连接。 `GatewayWsClient` 类型定义在 `src/gateway/server/ws-types.ts` 中，代表一个经过认证的 WebSocket 连接，包含身份信息、权限、事件订阅等元数据。

11.3.3 连接生命周期

每个 WebSocket 连接都经历以下生命周期：

阶段 1：连接建立

客户端建立 WebSocket 连接后，**第一个帧必须是 `connect` 请求**。这是 Gateway 协议的强制要求：

```
{
  "type": "req",
  "id": "1",
  "method": "connect",
  "params": {
    "auth": { "token": "my-gateway-token" },
    "clientInfo": {
      "name": "OpenClaw macOS",
      "version": "2026.2.6",
      "platform": "macos"
    }
  }
}
```

Gateway 收到 `connect` 后执行认证检查。认证失败则立即关闭连接并返回错误。

阶段 2：握手成功

认证通过后，Gateway 返回 `hello-ok` 响应，其中包含当前状态快照：

```
{
  "type": "res",
  "id": "1",
  "ok": true,
  "payload": {
    "type": "hello-ok",
    "presence": { /* 在线状态快照 */ },
    "health": { /* 健康状态快照 */ }
  }
}
```

客户端收到 `hello-ok` 后即进入就绪状态，可以发送请求和接收事件。

阶段 3：正常通信

在连接活跃期间，通信分为三种模式：

请求-响应（客户端 → Gateway → 客户端）：

```
客户端:  {"type":"req", "id":"42", "method":"status", "params":{}}
Gateway: {"type":"res", "id":"42", "ok":true, "payload":{...}}
```

服务器推送事件（Gateway → 客户端）：

```
Gateway: {"type":"event", "event":"agent", "payload":{...}, "seq":1}
Gateway: {"type":"event", "event":"presence", "payload":{...}}
Gateway: {"type":"event", "event":"tick", "payload":{...}}
```

Agent 流式事件（特殊的多帧响应）：

```
客户端:  {"type":"req", "id":"99", "method":"agent", "params":{"message":"Hello"}}
Gateway: {"type":"res", "id":"99", "ok":true, "payload":{"runId":"abc","status":"accept
Gateway: {"type":"event", "event":"agent", "payload":{"stream":"lifecycle","phase":"sta
Gateway: {"type":"event", "event":"agent", "payload":{"stream":"assistant","delta":"Hi
Gateway: {"type":"event", "event":"agent", "payload":{"stream":"assistant","delta":"the
Gateway: {"type":"event", "event":"agent", "payload":{"stream":"lifecycle","phase":"end
```

阶段 4：连接关闭

连接可能因多种原因关闭：

- 客户端主动断开
- 网络中断
- 认证过期
- Gateway 关闭/重启
- 心跳超时（长时间无活动）

Gateway 在连接关闭时清理该客户端的所有订阅和状态，并更新在线状态（Presence）信息。 `src/gateway/server-close.ts` 中的 `createGatewayCloseHandler` 处理优雅关闭逻辑。

11.3.4 帧协议格式

Gateway 的 WebSocket 帧协议基于 JSON 文本帧。三种帧类型：

请求帧

```
type RequestFrame = {
  type: "req";
  id: string;            // 请求 ID（客户端生成，用于匹配响应）
  method: string;         // 方法名
  params?: unknown;       // 方法参数
  idempotencyKey?: string; // 幂等键（防重复）
};
```

响应帧

```
type ResponseFrame = {
  type: "res";
  id: string;            // 对应请求的 ID
  ok: boolean;           // 是否成功
  payload?: unknown;     // 成功时的负载
  error?: {              // 失败时的错误
    code: string;
    message: string;
  };
};
```

事件帧

```
type EventFrame = {
  type: "event";
  event: string;          // 事件名
  payload: unknown;       // 事件负载
  seq?: number;           // 序列号（用于有序事件）
  stateVersion?: {        // 状态版本（用于增量更新）
    presence?: number;
    health?: number;
  };
};
```

幂等键（Idempotency Key） 是一个重要的安全机制。对于有副作用的操作（如 `send` 、 `agent` ），客户端必须提供一个唯一的幂等键。Gateway 维护一个短期的去重缓存——如果同一个幂等键的请求被发送两次（例如因为网络重试），第二次会直接返回第一次的结果，而不会重复执行。

帧验证：Gateway 对收到的每一帧进行 JSON Schema 验证。非 JSON 帧、格式不符的帧会导致连接立即关闭。第一帧必须是 `connect` ，否则也是硬关闭。这些都是 `docs/concepts/architecture.md` 中声明的不变量。

节点连接：节点（macOS/iOS/Android 设备）使用相同的 WebSocket 协议，但在 `connect` 请求中包含 `role: "node"` 以及设备能力声明（支持哪些命令、权限状态等）。